radius of 0, so they're invisible. Each element has a delay set to its array position `i` times 0.1 seconds (100 ms), after which the transition causes the circle to grow to a radius of 40 px. After each circle grows to that size, a second transition shrinks the circles to 20 px. The effect, which isn't easy to present with a screenshot, causes the circles to pulse sequentially.

3.2.3 DOM manipulation

Because these visual elements and buttons are all living in the DOM, it's important to know how to access and work with them both with D3 and using built-in JavaScript functionality.

Although D3 selections are extremely powerful, you sometimes want to deal specifically with the DOM element that's bound to the data. These DOM elements come with a rich set of built-in functionality in JavaScript. Getting access to the actual DOM element in the selection can be accomplished in one of two ways:

1. Using `this` in the inline functions
2. Using the `.node()` function

Inline functions always have access to the DOM element along with the datapoint and array position of that datapoint in the bound data. The DOM element, in this case, is represented by `this`. We can see it in action using the `.each()` function of a selection, which performs the same code for each element in a selection. We'll make a selection of one of our circles and then use `.each()` to send `d`, `i`, and `this` to the console to see what each corresponds to (which should look similar to the results in figure 3.7):

```
d3.select("circle").each(function(d,i) {
    console.log(d);console.log(i);console.log(this);
});
```

Unpacking this a bit, we can see the first thing echoed, `d`, is the data bound to the circle, which is a JSON object representing the Netherlands team. The second thing echoed, `i`, is the array position of that object in the array we used to create these elements, which in this case is 0 and means that `incomingData[0]` is the Netherlands JSON object. The last thing echoed to the console, `this`, is the `<circle>` DOM element itself.

We can also access this DOM element using the `.node()` function of a selection:

```
d3.select("circle").node();
```

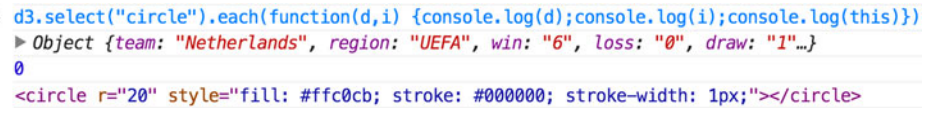


Figure 3.7 The console results of inspecting a selected element, which show first the datapoint in the selection, then its position in the array, and then the SVG element itself.